Uinsure Policies API
User guide

Run it, call it, test it and understand it. For the assessor, the developer who joins the code, and the person who calls the API.

Every command in this guide was run on the F7 branch on 2026-10-23, and every output, count and test number is copied from that run (`receipts.md`, next to this guide's source).

The big ideas

1. **Rules have IDs.** Every rule in the brief has an ID (SELL-2, CANCEL-1b) in `docs/requirements.md`. The same ID is a test trait, a code comment and the `ruleId` in an error body.
2. **Terms.** A policy is a list of terms. A renewal appends the next term and never overwrites dates, so a cancellation made in the last 30 days still works.
3. **One clock.** "Today" is the UK date, computed once from an injected `TimeProvider` in Europe/London. The domain only ever receives a `DateOnly`.
4. **One error shape.** Every failure is an RFC 9457 problem body. A 400, 404 or 409 is returned alone. A 422 lists every broken rule.
5. **One rounding.** Money is rounded in one method, `Money.Round`: 2 decimal places, half away from zero.
6. **Prove it.** 637 tests in four projects and a Postman collection of 253 requests, all green on this branch. Each rule has a unit test and an HTTP test.
7. **Evidence, not deployment.** The Bicep and the Azure DevOps pipeline were built and checked locally and never deployed.

1. What it is

A REST API that sells, retrieves, cancels, renews and records claims on home insurance policies.

It is the take-home for Uinsure's senior engineer role. The brief asked for the sell, retrieve, cancel and renew operations. All of its MUST, SHOULD and COULD rules are built, plus a cancellation quote and claims. Each rule has an ID and one agreed interpretation in `docs/requirements.md`. Where the brief was unclear, the interpretation is written down, not guessed.

▼ In depth

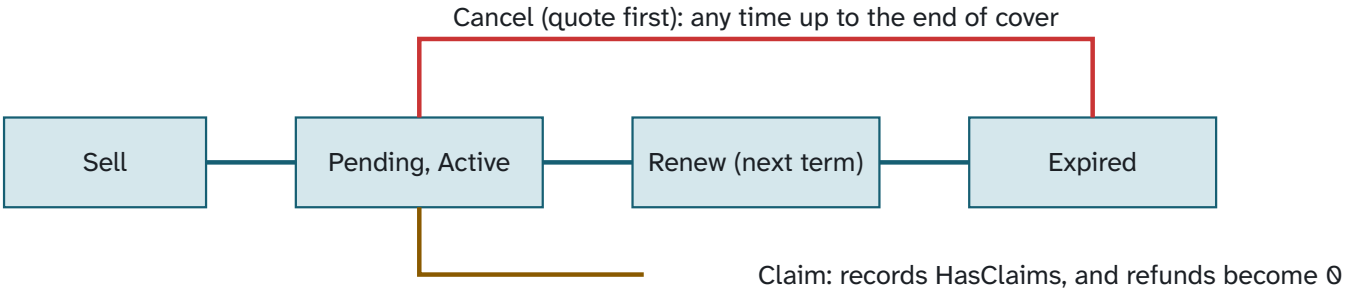


Figure 1. The operations on a policy's life. Status is derived from today, never stored.

The operations and the rules

Operation	Route	Rules
Sell	`POST /policies`	`SELL-1` to `SELL-7`
Retrieve	`GET /policies/{reference}`	`GET-1`
Cancel, and quote	`POST …/cancellation`, `GET …/cancellation-quote`	`CANCEL-1a` to `CANCEL-7`, `COULD-1`
Renew	`POST …/renewals`	`RENEW-1/2` to `RENEW-6`, `COULD-3`
Record a claim	`POST …/claims`	`COULD-2`, `COULD-2a`, `COULD-2b`, `COULD-2c`

The rules and their agreed interpretations are written in `docs/requirements.md`, one row per ID, and the controller that serves the six routes is a single class.

src/Uinsure.Policies.Api/Controllers/PoliciesController.cs:13

```
[ApiController]
[Route("policies")]
public sealed class PoliciesController(IPolicyService service) : ControllerBase
{
    [HttpPost]
    [EnableRateLimiting("writes")]
    public async Task<ActionResult<PolicyResponse>> Sell(SellPolicyRequest request,
CancellationToken cancellationToken)
```

Where the brief was unclear

The full list is the assumptions section of the README (README.md:151). The ones that change what you see:

Question in the brief	Taken as	Rule
Who chooses the reference?	The client. It is trimmed and upper-cased, and `abc-1` is `ABC-1`	SELL-1
"Up to 60 days in advance"	Start date from today to today + 60, inclusive. No backdating	SELL-2
One year in length	End = start + 1 year − 1 day. A 29 Feb start ends on 28 Feb	SELL-3
The cooling-off period	Runs to start + 14 days, inclusive. After it, the refund is pro rata over the term's real days (365 or 366)	CANCEL-1b CANCEL-1c
What is a renewal?	A new term on the same policy, starting the day after the end date	RENEW-4
Claims	Any claim sets `hasClaims` and makes every later refund £0	COULD-2

What is deliberately not done

> **Out of scope**
>
> 1. Authentication. The reference is the only key, so anyone who guesses it can read the policy or post a claim.
> 2. Pricing (the client sends every amount) and real payment processing.
> 3. Live deployment. The Azure files are evidence (section 6).

The known limits, including the security scope, are in `docs/design.md` and the README's "Known limits".

Check your understanding

Question	Answer
A1. What does it do, and does it meet the brief?	Sell, retrieve, cancel (with a quote), renew and claims. Every MUST, SHOULD and COULD rule has an ID in `docs/requirements.md`.
A5. Where was the brief unclear?	The assumptions table above, and one interpretation per rule in the requirements file.
A9. What is not done?	Authentication, pricing, payment processing and live deployment.
U10. Do I need a key?	No. There is no authentication, which is a known gap that the security scope names.

2. Run it

Install the .NET 10 SDK, build, run: no database setup.

Locally the API uses SQLite and creates its own schema at start-up. In Azure it uses SQL Server, chosen by two configuration keys. The steps below were run in order; the outputs are what that run printed.

▼ In depth

1. Check the SDK. `global.json` pins `10.0.100` with `rollForward` set to `latestFeature`, so any 10.0.1xx or later feature band works.

```
dotnet --version
```

You should see: The installed version, here:

```
10.0.402
```

2. Build with warnings as errors, as the pre-commit hook does.

```
dotnet build Uinsure.Policies.slnx -warnaserror
```

You should see: No warnings and no errors. The last lines of the run:

```
Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:12.14
```

3 Run the API on port 5080 with the `http` launch profile (Development, so SQLite).

```
dotnet run --project src/Uinsure.Policies.Api --launch-profile http
```

You should see: The start-up lines:

```
      Now listening on: http://localhost:5080
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
```

4 Check health from a second terminal. `/health/live` runs no checks. `/health/ready` runs the checks tagged `ready`, which here is the database.

```
curl -s -ihttp://localhost:5080/health/live
curl -s -i http://localhost:5080/health/ready
```

You should see: Both answer 200 with a plain-text body:

```
HTTP/1.1 200 OK
Content-Type: text/plain

Healthy
```

5 Open the API description. In Development only, OpenAPI is at `/openapi/v1.json` and the Scalar page at `/scalar`. Both answered 200 in the run.

```
curl -s -i http://localhost:5080/openapi/v1.json -o /dev/null -w "openapi
status %{http_code}\n"
curl -s -o /dev/null -w "scalar status %{http_code}\n"
http://localhost:5080/scalar/v1
```

You should see: The run printed:

```
openapi status 200
scalar status 200
```

The SQLite and SQL Server switch

One `PoliciesDbContext` serves both. `Database:Provider` (`Sqlite` or `SqlServer`) chooses the provider and `ConnectionStrings:Policies` holds the connection. The development file sets SQLite and a database file per checkout (`policies.development.db`, ignored by git).

src/Uinsure.Policies.Persistence/PersistenceServiceCollectionExtensions.cs:18

```
        services.AddDbContext<PoliciesDbContext>((serviceProvider, options) =>
        {
            var connectionString = configuration.GetConnectionString("Policies");
            _ = serviceProvider.GetRequiredService<IOptions<DatabaseOptions>>
().Value.ProviderKind switch
            {
                DatabaseProvider.Sqlite => options.UseSqlite(connectionString),
                DatabaseProvider.SqlServer => options.UseSqlServer(connectionString),
                _ => throw new InvalidOperationException("Unknown database
provider."),
            };
        });
```

> **No SQL Server here**
> No test runs against SQL Server, and this guide ran none. What is shown for it is the configuration above and the generated migration script below.

Migrations

With SQLite, the schema is created from the model at start-up (`EnsureCreated`, src/Uinsure.Policies.Persistence/PersistenceServiceCollectionExtensions.cs:37). The app never calls `Migrate()`. The migration, `InitialCreate`, is for SQL Server only and is applied by the pipeline as a script.

1. Generate the idempotent script, as the pre-push hook does.

   ```
   dotnet ef migrations script --idempotent --project
   src/Uinsure.Policies.Persistence --startup-project src/Uinsure.Policies.Api -
   o migrate.sql
   ```

   You should see: The tool wrote a 242-line T-SQL script and printed one warning about its own version:

   ```
   The Entity Framework tools version '9.0.9' is older than that of the runtime
   '10.0.13'.
   ```

   ```
   IF OBJECT_ID(N'[__EFMigrationsHistory]') IS NULL
   BEGIN
       CREATE TABLE [__EFMigrationsHistory] (
           [MigrationId] nvarchar(150) NOT NULL,
           [ProductVersion] nvarchar(32) NOT NULL,
           CONSTRAINT [PK___EFMigrationsHistory] PRIMARY KEY ([MigrationId])
       );
   END;
   GO
   ```

Visual Studio

1. Open `Uinsure.Policies.slnx`, set **Uinsure.Policies.Api** as the startup project, choose the **http** launch profile and press Ctrl+F5.

   You should see: A browser opens on `/health/live` and shows `Healthy`. Only one process can hold port 5080, so stop a terminal run first.

PLACEHOLDER: screenshot to be taken by hand

Visual Studio: startup project and the http profile (figure 2)

Figure 2. Visual Studio's startup toolbar with Uinsure.Policies.Api and the http profile chosen. Placeholder: no numbered markers until the real image exists.

> **Not on this branch**
> The UI plan's launch profile that starts the API and the web app together (U9) is not on this branch, so this guide gives the API-only profile. The UI guide covers the combined launch.

Check your understanding

Question	Answer
A2. How do I build and run it quickly?	SDK 10, `dotnet build`, `dotnet run --project src/Uinsure.Policies.Api --launch-profile http`. SQLite needs no setup.
D8. How does the database get its schema?	`EnsureCreated` on SQLite at start-up. EF migrations, applied as a script, for SQL Server. The app never calls `Migrate()`.
A7. Why SQLite locally and SQL Server in Azure?	One context, and the provider is a configuration key. SQLite needs nothing installed.

3. Use it, endpoint by endpoint

Six routes under `/policies`. Every failure is a problem body that names the rule broken.

The examples below were sent with `curl` to the API started in section 2, on 2026-10-23, so "today" is 2026-10-23. Each endpoint gives the request, the response, its failures, and the Postman request that does the

same. A `…` inside a command or an output marks text left out of this page; the full text is in `receipts.md`. The sale of GUIDE-002 (1,200.00, Direct Debit, starting today) is in R5b there.

▼ In depth

Sell a policy

SELL-1 to **SELL-7**. `POST /policies` takes the reference you choose, the type (`Household` or `BuyToLet`), the start date, the amount, `autoRenew`, the payment method (`Card`, `DirectDebit`, `Cheque`), one to three policyholders and the property. The end date is calculated.

1. Sell a policy that starts 2026-11-01.

```
curl -s -i-X POST http://localhost:5080/policies -H "Content-Type:
application/json" -d '{"reference":"GUIDE-
001","type":"Household","startDate":"2026-11-
01","amount":803.40,"autoRenew":true,"paymentMethod":"Card","policyholders":
[{"firstName":"Ada","lastName":"Lovelace","dateOfBirth":"1985-12-
10"}],"property":{"addressLine1":"1 Example
Street","addressLine2":"Exampleton","postcode":"EX1 1AA"}}'
```

You should see: 201 with a `Location` header and the whole policy. The end date, the term and the payment were calculated.

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Location: /policies/GUIDE-001

{"reference":"GUIDE-
001","type":"Household","status":"Pending","currentTermNumber":1,"startDate":
"2026-11-01","endDate":"2027-10-
31","amount":803.40,"hasClaims":false,"autoRenew":true,"policyholders":
[{"firstName":"Ada","lastName":"Lovelace","dateOfBirth":"1985-12-
10"}],"property":{"addressLine1":"1 Example
Street","addressLine2":"Exampleton","addressLine3":null,"postcode":"EX1
1AA"},"terms":[{"termNumber":1,"startDate":"2026-11-01","endDate":"2027-10-
31","amount":803.40,"paymentReference":"PAY-
180270d7cc1c4bd7ba00e24de5b00b1b"}],"payments":[{"reference":"PAY-
180270d7cc1c4bd7ba00e24de5b00b1b","type":"Card","amount":803.40}],"refunds":
[],"claims":[],"cancellation":null}
```

2. Send the same reference again.

```
curl -s -i-X POST http://localhost:5080/policies -H "Content-Type:
application/json" -d '…the same body…'
```

You should see: 409: `SELL-1`. A 409 is returned alone, before any 422 rule is looked at.

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json; charset=utf-8

{"type":"/problems/sell-1","title":"Policy reference already
exists","status":409,"detail":"A policy with reference 'GUIDE-001' already
exists.","instance":"/policies","traceId":"00-
35dbb41576bf3a12eb853c86d9b85070-a4bbb08391adcac3-00","ruleId":"SELL-
1","reference":"GUIDE-001"}
```

3. Send a sale with the start date in the past and an amount of `0`.

```
curl -s -i-X POST http://localhost:5080/policies -H "Content-Type:
application/json" -d '{"reference":"GUIDE-
003","type":"Household","startDate":"2026-10-01","amount":0,…}'
```

You should see: 422 with *both* broken rules in `violations[]`. `ruleId` is the first of them in requirements order.

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json; charset=utf-8

{"type":"/problems/sell-2","title":"Start date outside the sale
window","status":422,"detail":"2 rules
failed.","instance":"/policies","traceId":"00-
247ad89a1cb98197b9d4db31f6821a2d-0d65be4b8c3254f6-00","ruleId":"SELL-
2","violations":[{"ruleId":"SELL-2","detail":"The start date must be between
2026-10-23 and 2026-12-22."},{"ruleId":"SELL-7","detail":"The policy amount
must be greater than 0."}]}
```

4 Send a body with only `{"reference":"GU"}`.

```
curl -s -i-X POST http://localhost:5080/policies -H "Content-Type:
application/json" -d '{"reference":"GU"}'
```

You should see: 400: a shape error. It has an `errors` map and no `ruleId`.

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json; charset=utf-8

{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.1","title":"One or
more validation errors
occurred.","status":400,"instance":"/policies","errors":{"type":["The Type
field is required."],"amount":["The Amount field is required."],"property":
["The Property field is required."],"autoRenew":["The AutoRenew field is
required."],"startDate":["The StartDate field is required."],"paymentMethod":
["The PaymentMethod field is required."],"policyholders":["The Policyholders
field is required."]},"traceId":"00-a5eb6750da9764cda9cbdbd1bc236837-
14df225b286f368b-00"}
```

Failure	Status	Rule
Reference already used (any case)	409	`SELL-1`
Start date not between today and today + 60	422	`SELL-2`
Not 1 to 3 policyholders; one is under 16 on the start date	422	`SELL-4` `SELL-5`
Amount 0 or below	422	`SELL-7`
Missing field, unknown field, wrong type, reference not 3 to 20 letters, digits or hyphens	400	none (shape)

In Postman: folder **F1 Sell**, request `SELL-1 · duplicate reference → 409`.

Retrieve a policy

`GET-1`. `GET /policies/{reference}` returns the policy, its terms, policyholders, property, payments, refunds and claims. The reference in the path is trimmed and upper-cased, so a lower-case one finds it. `status` is derived from today: `Pending` before the first start, `Active`, `Expired` after the last end, and `Cancelled`, which overrides the rest. The policy-level dates and amount describe the current term.

1. Read it back, using a lower-case reference.

```
curl -s -ihttp://localhost:5080/policies/guide-001
```

You should see: 200 with the same body as the sale response above. The status line and the first fields:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"reference":"GUIDE-
001","type":"Household","status":"Pending","currentTermNumber":1,"startDate":
"2026-11-01","endDate":"2027-10-
31","amount":803.40,"hasClaims":false,"autoRenew":true, …
```

2. Ask for a reference that does not exist.

```
curl -s -ihttp://localhost:5080/policies/NO-SUCH-POLICY
```

You should see: 404: `GET-1`.

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json; charset=utf-8

{"type":"/problems/get-1","title":"Policy not
found","status":404,"detail":"No policy with reference 'NO-SUCH-
POLICY'.","instance":"/policies/NO-SUCH-POLICY","traceId":"00-
58d2590d72cf26ebb8f68a942433649a-16b49e0ead98e59b-00","ruleId":"GET-
1","reference":"NO-SUCH-POLICY"}
```

3. Add a query key the route does not bind.

```
curl -s -i"http://localhost:5080/policies/GUIDE-001?reference=X"
```

You should see: 400. Unknown query keys are refused. A key named like a route parameter counts as unknown.

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json; charset=utf-8

{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.1","title":"One or
more validation errors occurred.","status":400,"instance":"/policies/GUIDE-
001","errors":{"reference":["Unknown query parameter."]},"traceId":"00-
36aa14f3cb34839de8f2a0d87fbb67b1-cba103a7aeeda6a8-00"}
```

In Postman: folder **F2 Retrieve**, request `GET-1 · unknown reference → 404`.

Quote, then cancel

`COULD-1`, `CANCEL-1a` to `CANCEL-7`. Quote first: `GET …/cancellation-quote?cancellationDate=yyyy-MM-dd` changes nothing. Then `POST …/cancellation`; the body is optional, and `cancellationDate` defaults to today. Each refund goes back to the method of the payment it reverses (`CANCEL-2`).

Cancelled on	`ruleApplied`	Refund
Before the start date	`CANCEL-1a`	Full amount
Start date to start + 14 days, inclusive	`CANCEL-1b`	Full amount
Later	`CANCEL-1c`	Amount × unused days ÷ days in the term
The policy has a claim	`COULD-2`	0.00, whatever the date
The term was never paid (an unpaid renewal)	`UNPAID-TERM`	0.00, and it wins over a claim

1. Quote a cancellation on 2026-11-10, nine days after the start.

```
curl -s -i"http://localhost:5080/policies/GUIDE-001/cancellation-quote?cancellationDate=2026-11-10"
```

You should see: 200: cooling-off, so the full 803.40 back to the Card.

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"reference":"GUIDE-001","cancellationDate":"2026-11-10","ruleApplied":"CANCEL-1b","totalRefund":803.40,"terms":[{"termNumber":1,"ruleApplied":"CANCEL-1b","daysInTerm":365,"daysUsed":10,"unusedDays":355,"refundAmount":803.40}],"refunds":[{"type":"Card","amount":803.40,"paymentReference":"PAY-180270d7cc1c4bd7ba00e24de5b00b1b"}],"quotedOn":"2026-10-23"}
```

2 Quote 2027-04-30, after cooling-off.

```
curl -s -i"http://localhost:5080/policies/GUIDE-001/cancellation-quote?
cancellationDate=2027-04-30"
```

You should see: 200: pro rata. 181 days used, 184 unused, 803.40 × 184 ÷ 365 = 405.00.

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"reference":"GUIDE-001","cancellationDate":"2027-04-
30","ruleApplied":"CANCEL-1c","totalRefund":405.00,"terms":
[{"termNumber":1,"ruleApplied":"CANCEL-
1c","daysInTerm":365,"daysUsed":181,"unusedDays":184,"refundAmount":405.00}],
"refunds":[{"type":"Card","amount":405.00,"paymentReference":"PAY-
180270d7cc1c4bd7ba00e24de5b00b1b"}],"quotedOn":"2026-10-23"}
```

3 Cancel for 2026-11-10.

```
curl -s -i-X POST http://localhost:5080/policies/GUIDE-001/cancellation -H
"Content-Type: application/json" -d '{"cancellationDate":"2026-11-10"}'
```

You should see: 201, no `Location`. The answer agrees with the quote and now carries a refund `reference`.

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8

{"reference":"GUIDE-001","cancellationDate":"2026-11-
10","ruleApplied":"CANCEL-1b","totalRefund":803.40,"terms":
[{"termNumber":1,"ruleApplied":"CANCEL-
1b","daysInTerm":365,"daysUsed":10,"unusedDays":355,"refundAmount":803.40}],"
refunds":[{"reference":"REF-
82c873b208b34aedb5d6ecde150ddf10","type":"Card","amount":803.40,"paymentRefer
ence":"PAY-180270d7cc1c4bd7ba00e24de5b00b1b"}]}
```

4 Cancel again.

```
curl -s -i-X POST http://localhost:5080/policies/GUIDE-001/cancellation
```

You should see: 409: `CANCEL-4`. The body gives the original cancellation date.

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json; charset=utf-8

{"type":"/problems/cancel-4","title":"Policy already
cancelled","status":409,"detail":"The policy was already cancelled on 2026-
11-10.","instance":"/policies/GUIDE-001/cancellation","traceId":"00-
a9d78d5db903240285b4d43b48543cbc-ad5eca82de206571-00","ruleId":"CANCEL-
4","cancellationDate":"2026-11-10"}
```

5 Cancel a different policy for yesterday.

```
curl -s -i-X POST http://localhost:5080/policies/GUIDE-002/cancellation -H
"Content-Type: application/json" -d '{"cancellationDate":"2026-10-22"}'
```

You should see: 422: `CANCEL-3`. No past dates.

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json; charset=utf-8

{"type":"/problems/cancel-3","title":"Cancellation date in the
past","status":422,"detail":"The cancellation date cannot be in the
past.","instance":"/policies/GUIDE-002/cancellation","traceId":"00-
c2f67c1eb0a0ef7fe6d5bedfb01d5b1a-f90b9a508fffe405-00","ruleId":"CANCEL-
3","today":"2026-10-23","violations":[{"ruleId":"CANCEL-3","detail":"The
cancellation date cannot be in the past."}]}
```

Failure	Status	Rule
Policy already cancelled	409	`CANCEL-4`
Date before today; date after the end of cover	422	`CANCEL-3` `CANCEL-7`
Unknown reference	404	`GET-1`
Quote without `cancellationDate`, a bad date format, an unknown query key, or an unknown JSON field	400	none (shape)

In Postman: folder **F3 Cancel**, requests `COULD-1 · quote then cancel for the same date agree → 201` and `CANCEL-4 · cancel twice → 409`.

Record a claim

`COULD-2`, `COULD-2a` to `COULD-2c`. `POST …/claims` takes a required `claimDate` and an optional `description` of up to 500 characters. The date must not be in the future and must fall inside a term's cover.

1. Sell a policy starting today (GUIDE-002, 1,200.00 by Direct Debit), then claim for today.

```
curl -s -i-X POST http://localhost:5080/policies/GUIDE-002/claims -H
"Content-Type: application/json" -d '{"claimDate":"2026-10-
23","description":"Burst pipe"}'
```

You should see: 201 with `Location` pointing at the policy, not at a claim.

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Location: /policies/GUIDE-002

{"reference":"CLM-GUIDE-002-1","policyReference":"GUIDE-
002","claimDate":"2026-10-23","description":"Burst pipe","hasClaims":true}
```

2. Quote a cancellation for 2027-04-30 on that policy.

```
curl -s -i"http://localhost:5080/policies/GUIDE-002/cancellation-quote?
cancellationDate=2027-04-30"
```

You should see: 200 and nothing back: `COULD-2`. No `refunds` are listed, because no £0 refund record is ever created.

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"reference":"GUIDE-002","cancellationDate":"2027-04-
30","ruleApplied":"COULD-2","totalRefund":0.00,"terms":
[{"termNumber":1,"ruleApplied":"COULD-
2","daysInTerm":365,"daysUsed":190,"unusedDays":175,"refundAmount":0.00}],"re
funds":[],"quotedOn":"2026-10-23"}
```

3 Claim for tomorrow.

```
curl -s -i-X POST http://localhost:5080/policies/GUIDE-002/claims -H
"Content-Type: application/json" -d '{"claimDate":"2026-10-24"}'
```

You should see: 422: `COULD-2a`.

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json; charset=utf-8

{"type":"/problems/could-2a","title":"Claim date in the
future","status":422,"detail":"The claim date cannot be in the
future.","instance":"/policies/GUIDE-002/claims","traceId":"00-
ec8b21a601d7b77acf50a481648bfb42-6173c6107c515c84-00","ruleId":"COULD-
2a","today":"2026-10-23","violations":[{"ruleId":"COULD-2a","detail":"The
claim date cannot be in the future."}]}
```

4 Claim on the policy that was cancelled earlier (GUIDE-001).

```
curl -s -i-X POST http://localhost:5080/policies/GUIDE-001/claims -H
"Content-Type: application/json" -d '{"claimDate":"2026-11-10"}'
```

You should see: 409: `COULD-2c`.

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json; charset=utf-8

{"type":"/problems/could-2c","title":"Claim on a cancelled
policy","status":409,"detail":"The policy was cancelled on 2026-11-10, so a
claim cannot be recorded.","instance":"/policies/GUIDE-
001/claims","traceId":"00-12f616a40a6b6fef44134544f72526c7-e8072e5c20eb56f1-
00","ruleId":"COULD-2c","cancellationDate":"2026-11-10"}
```

> **A claim cannot be undone**
> It sets `hasClaims` for good, and every later refund is £0. A claim is not idempotent: sending it twice records two. That is why the guide shows it only on a throw-away policy.

Failure	Status	Rule
Claim date after today; outside every term's cover	422	`COULD-2a` `COULD-2b`
Policy is cancelled	409	`COULD-2c`
No `claimDate`; description over 500 characters	400	none (shape)

In Postman: folder **F3 Cancel**, request `COULD-2 · claim then cancel refunds nothing → 201`.

Renew a policy

`RENEW-1/2` to `RENEW-6`, `COULD-3`. `POST …/renewals` takes the new term's `amount` and, optionally, a `paymentMethod`. You may renew from 30 days before the latest term's end date to the end date, both inclusive. The new term starts the day after the end. With `autoRenew` on, the payment method defaults to the current term's, and a Cheque is refused. With it off, the term is created unpaid and a `paymentMethod` is refused.

1. Renew the policy sold today (GUIDE-002), which ends 2027-10-22.

```
curl -s -i-X POST http://localhost:5080/policies/GUIDE-002/renewals -H
"Content-Type: application/json" -d '{"amount":1250.00}'
```

You should see: 422: `RENEW-1/2`. The window is named in the body.

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json; charset=utf-8

{"type":"/problems/renew-1-2","title":"Outside the renewal
window","status":422,"detail":"A policy can be renewed from 2027-09-22 to
2027-10-22.","instance":"/policies/GUIDE-002/renewals","traceId":"00-
213bda48b9797f9d89d00d5dc2d03ae6-7a8067215543b9d2-00","ruleId":"RENEW-
1/2","windowStart":"2027-09-22","windowEnd":"2027-10-22","violations":
[{"ruleId":"RENEW-1/2","detail":"A policy can be renewed from 2027-09-22 to
2027-10-22."}]}
```

> **A successful renewal is not shown live**
> A policy can only be sold from today, so it cannot be inside its renewal window for 335 days, and the API runs on the real clock. The acceptance tests set the clock instead. On this branch the 13 tests traited `RENEW-4` passed (`dotnet test --solution Uinsure.Policies.slnx --no-build -- --filter-trait "Rule=RENEW-4"`). A renewal returns the policy with a second entry in `terms` . Postman's **F4 Renew** has no happy-path requests for the same reason.

Failure	Status	Rule
Outside the window; amount 0 or below	422	`RENEW-1/2` `RENEW-5`
`paymentMethod` sent with auto-renew off; a Cheque with auto-renew on	422	`RENEW-3b` `COULD-3`
Policy is cancelled	409	`RENEW-6`
No body, no amount, a start date sent (the client does not choose it), a bad payment method	400	none (shape)

In Postman: folder **F4 Renew**, request `RENEW-3b · auto-renew off with a paymentMethod, today → 422` .

Reading an error

Every body follows RFC 9457. `type` is `/problems/` and the rule ID in lower case, with `/` as `-`. `ruleId` names the rule, `violations[]` lists every broken rule on a 422, and `traceId` finds the request in the logs. One place maps a rule to its status, type and title.

src/Uinsure.Policies.Api/Errors/ProblemCatalog.cs:6

```
// The one place a rule ID becomes an HTTP status, a problem type and a title.
internal static class ProblemCatalog
{
    private static readonly Dictionary<string, string> Titles = new()
    {
        [RuleIds.Get1] = "Policy not found",
        [RuleIds.Sell1] = "Policy reference already exists",
```

Status	What kind of problem	Carries
400	The request has the wrong shape: a missing or unknown field, a wrong type, a bad date, an unknown query key	`errors` map, `traceId`, no `ruleId`
404	Unknown reference (`GET-1`)	`ruleId`
409	State conflict: `SELL-1`, `CANCEL-4`, `RENEW-6`, `COULD-2c`. Also two writes on one policy (`/problems/concurrency`, no rule)	`ruleId` (except concurrency)
422	A business rule is broken	`ruleId` and `violations[]`
429	Rate limit per client IP; `/problems/rate-limited`	`Retry-After` header
500	Anything unexpected	Generic `detail` and `traceId`. The exception goes to the logs only

> **Evaluation order**
> 400 shape, then 404, then 409, each returned alone. Only then are all the 422 rules checked together and listed. So a duplicate reference with a bad start date is a 409, and a bad start date with a bad amount is one 422 with two violations.

A 429, a 500, and a 413 for a body over 65,536 bytes are covered by the acceptance tests and were not provoked live for this guide.

Check your understanding

Question	Answer
U1. How do I sell a policy?	`POST /policies` with a reference, start date, amount, payment method, policyholders and property.

Question	Answer
U2. How do I look one up, and what does its status mean?	`GET /policies/{reference}`. Status is derived from today: Pending, Active, Expired, or Cancelled.
U3. How do I cancel, and how much comes back?	Quote with `GET …/cancellation-quote`, then `POST …/cancellation`. Full refund to start + 14, pro rata after, `0` with a claim.
U4. When can I renew?	From 30 days before the end date to the end date.
U5. What does a claim do?	It sets `hasClaims`, and every later refund is £0.
U6/U7. How do I read an error, and what does the status say?	Read `ruleId` and `violations[]`. 400 shape, 404 unknown, 409 conflict, 422 broken rule, 429 slow down, 500 not you.
U9. Why was my duplicate or backdated request refused?	A duplicate reference (any case) is 409 **`SELL-1`**. A start date before today or after today + 60 is 422 **`SELL-2`**.
D6. How do errors get their shape?	`ProblemCatalog` and `ResultExtensions` decide it in one place.

4. Run the tests yourself

Four ways to run the same checks: Postman app, Postman CLI, Visual Studio, and `dotnet test`.

The .NET tests need nothing running. The Postman collection needs the API on port 5080. All 637 .NET tests and all 253 Postman requests passed on this branch on 2026-10-23. The web suites (Vitest and Playwright) belong to the UI guide, and are not run or shown here because the web app is not on this branch.

▼ In depth

MAN-4: dotnet test, from the command line

1. Run everything.

```
dotnet test --solution Uinsure.Policies.slnx
```

You should see: Four projects pass, then the totals:

```
Test run summary: Passed!

tests\Uinsure.Policies.Services.Tests\bin\Debug\net10.0\Uinsure.Policies.Serv
ices.Tests.dll (net10.0|x64) passed (906ms)

tests\Uinsure.Policies.Architecture.Tests\bin\Debug\net10.0\Uinsure.Policies.
Architecture.Tests.dll (net10.0|x64) passed (1s 511ms)

tests\Uinsure.Policies.Domain.Tests\bin\Debug\net10.0\Uinsure.Policies.Domain
.Tests.dll (net10.0|x64) passed (921ms)

tests\Uinsure.Policies.Api.Tests\bin\Debug\net10.0\Uinsure.Policies.Api.Tests
.dll (net10.0|x64) passed (10s 319ms)

  total: 637
  failed: 0
  succeeded: 637
  skipped: 0
  duration: 10s 753ms
```

2. Run one project.

```
dotnet test --project tests/Uinsure.Policies.Domain.Tests
```

You should see: Only that project's totals. The four runs gave:

```
Domain.Tests    total: 173  failed: 0  succeeded: 173  duration: 1s 062ms
Services.Tests  total: 65   failed: 0  succeeded: 65   duration: 1s 077ms
Api.Tests       total: 332  failed: 0  succeeded: 332  duration: 9s 476ms
Architecture    total: 67   failed: 0  succeeded: 67   duration: 1s 482ms
```

3 Run one rule by its trait.

```
dotnet test --solution Uinsure.Policies.slnx -- --filter-trait "Rule=CANCEL-1b"
```

You should see: Eight tests across the projects, all passed:

```
total: 8
failed: 0
succeeded: 8
skipped: 0
duration: 2s 923ms
```

> **Careful**
> The filter flag comes after `--`, and the option is `--filter-trait` (xUnit v3 on Microsoft.Testing.Platform). The older `--filter "Rule=…"` form is for VSTest and is not what this solution runs on.

What each project proves

Project	Tests	Proves	Needs
`Domain.Tests`	173	Every rule at both sides of its boundary, with `DateOnly today` passed in	Nothing
`Services.Tests`	65	Orchestration and `UkCalendar`, including the BST boundary	Nothing
`Api.Tests`	332	The acceptance suite over HTTP: `WebApplicationFactory`, SQLite in memory, a fake clock	Nothing
`Architecture.Tests`	67	Layering rules and rule coverage	Nothing

Rule coverage

Each test carries `[Trait("Rule", "<id>")]`. `RuleCoverageTests` fails if a rule ID in `docs/requirements.md` lacks a unit trait or an integration trait, unless it is listed as pending.

tests/Uinsure.Policies.Api.Tests/CancelCoolingOffTests.cs:22

```
    // 2026-10-17 is both the start date and today (CANCEL-3 allows today): 1 day
used, 289.40 back to the Cheque.
    [Fact]
    [Trait("Rule", "CANCEL-1b")]
    [Trait("Rule", "CANCEL-2")]
    [Trait("Rule", "CANCEL-3")]
    public async Task Cancel_on_the_start_date_refunds_in_full_to_the_Cheque_paid()
    {
        const string reference = "CAN1B-START";
        var sold = new SellPolicyRequestBuilder(reference)

.WithStartDate(TestClock.Today).WithAmount(289.40m).WithPaymentMethod("Cheque").Build
();
```

1. Run the coverage tests alone.

   ```
   dotnet test --project tests/Uinsure.Policies.Architecture.Tests -- --filter-
   class "*RuleCoverageTests"
   ```

   You should see: Eleven coverage tests passed:

   ```
   total: 11
   failed: 0
   succeeded: 11
   ```

MAN-2: the Postman CLI

1. Start the API (section 2, step 3). Then, from the repository root, run the whole collection. No login is needed; the CLI warns that publishing to the Postman cloud needs one.

   ```
   postman collection run tests/postman/Uinsure.Policies.postman_collection.json
   -e tests/postman/local.postman_environment.json
   ```

   You should see: Each request, then each assertion as `Pass`, then the summary table. Failures would appear in the `failed` column.

   ```
   |              iterations |                   1 |                   0 |
   |                requests |                 253 |                   0 |
   |            test-scripts |                 109 |                   0 |
   |      prerequest-scripts |                 114 |                   0 |
   |              assertions |                 246 |                   0 |
   | total run duration: 17.7 s                                            |
   | average response time: 21 ms [min: 11 ms, max: 76 ms, s.d.: 7 ms] |
   ```

2. Run one folder with `-i`.

```
postman collection run tests/postman/Uinsure.Policies.postman_collection.json
-e tests/postman/local.postman_environment.json -i "F3 Cancel"
```

You should see: Only that folder's requests:

```
|                 requests |                  87 |                   0 |
|            test-scripts |                  33 |                   0 |
|      prerequest-scripts |                  33 |                   0 |
|              assertions |                  82 |                   0 |
| total run duration: 5.1 s                                              |
```

> **Reading the numbers**
> The summary counts *requests* (253, including the set-up sales and read-backs), *test scripts* (109, the requests that assert something) and *assertions* (246). The README and design notes call the 109 "requests"; the CLI shows they are test scripts.

MAN-1: the Postman app

1. Install Postman Desktop. The browser version cannot reach `localhost` without the Desktop Agent.

2. Choose **Import** and select both files in `tests/postman/`: `Uinsure.Policies.postman_collection.json` and `local.postman_environment.json`.

   You should see: The collection **Uinsure.Policies** with six folders, and an environment named **local** (figure 3).

3. Pick **local** in the environment dropdown, top right. It holds one variable, `baseUrl = http://localhost:5080`. Start the API.

4. Run the lot: on the collection, ··· → **Run collection**, then **Run**. Run a folder with ··· → **Run folder**. Run one request by opening it and choosing **Send**, then reading the **Test Results** tab.

   You should see: The runner lists each request with its passed assertions; the CLI run above reports 246 assertions for the same collection (figure 4).

PLACEHOLDER: screenshot to be taken by hand

Postman: collection imported, environment "local" selected

Figure 3. Postman after the import. Placeholder: markers are placed from the real image, so none are drawn yet.

PLACEHOLDER: screenshot to be taken by hand

Postman: Collection Runner results

Figure 4. The Collection Runner after a full run. Placeholder.

Every request stands alone: its pre-request script makes a fresh reference and today's dates and sells whatever it needs, so order does not matter and nothing needs resetting.

Folder	Covers	Rule IDs
F0b Walking skeleton	Health, live and ready	none
F1 Sell	Happy path and violations of the sale	`SELL-1` to `SELL-7`, shape checks
F1b Layered models	Amount scale and limits, round trips through storage	`GET-1`, shape checks
F2 Retrieve	Read-back, status, 404	`GET-1`
F3 Cancel	Quote, cancel, refunds, claims	`CANCEL-1a` to `CANCEL-7`, `COULD-1`, `COULD-2` to `COULD-2c`
F4 Renew	Renewal failures on the real clock	`RENEW-1/2` to `RENEW-6`, `COULD-3`

MAN-3: Visual Studio

1. Open `Uinsure.Policies.slnx`. For every folder on disk, use **Switch Views → Folder View**.

2. Open **Test** → **Test Explorer** and choose **Run All Tests** (Ctrl+R, A).

   You should see: The tests group by project. Passing tests show a green tick; the .NET run above had 637 (figure 5).

3. Run one project, class or test: right-click it and choose **Run**. To run one rule, use **Group By** → **Traits**: the `Rule` trait lists tests by rule ID, and you run the group.

4. Debug a test: right-click it and choose **Debug**. Read a failure by selecting the test: the detail pane shows the message and stack, and the assertion line is a link.

PLACEHOLDER: screenshot to be taken by hand

Visual Studio: Test Explorer, grouped by Rule trait

Figure 5. Test Explorer grouped by the Rule trait. Placeholder.

> **Not run here**
> The Visual Studio steps come from the README and were not run for this guide. The Test Explorer behaviour is Visual Studio's; this guide has no run of it to quote.

The harness self-test, and the git hooks

The repository's own hooks guard every commit and push. **Pre-commit**: refuses commits on a locked `main`, scans the staged files for private content, and builds with warnings as errors. **Pre-push**: refuses pushes to `main`, then builds in Release, runs the whole suite, generates the idempotent migration script, and runs the harness self-test.

`.githooks/pre-push:25`

```
if [ -f Uinsure.Policies.slnx ]; then
  dotnet build Uinsure.Policies.slnx -c Release -warnaserror -nologo -v quiet || {
echo "pre-push: build failed" >&2; exit 1; }
  dotnet test --solution Uinsure.Policies.slnx -c Release --no-build || { echo "pre-
push: tests failed" >&2; exit 1; }
  out="${TMPDIR:-${TEMP:-/tmp}}/uinsure-migrate.sql"
  dotnet tool restore >/dev/null && dotnet ef migrations script --idempotent \
    --project src/Uinsure.Policies.Persistence --startup-project
src/Uinsure.Policies.Api -o "$out" \
    || { echo "pre-push: idempotent migration script failed" >&2; exit 1; }
```

1. Run the harness self-test, which checks the guard hooks themselves.

```
node --test ".claude/hooks/__tests__/*.test.mjs"
```

You should see: 221 tests, none failing. It is the slowest check here:

```
1..221
# tests 221
# suites 0
# pass 221
# fail 0
# cancelled 0
# skipped 0
# todo 0
# duration_ms 100183.7204
```

MAN-7: which runner, when

Runner	Tests	Use when	Command or click path	Needs first	Measured
`dotnet test`	All 637 .NET tests	Before any commit; in CI	`dotnet test --solution Uinsure.Policies.slnx`	Nothing	10.7 s test time; 16.6 s wall
One rule by trait	Tests of one rule ID	Working on one rule	`… -- --filter-trait "Rule=CANCEL-1b"`	Nothing	2.9 s (8 tests)
Postman CLI	253 requests, 246 assertions, against the live API	Check a running build as a client	`postman collection run …`	API on 5080	17.7 s
Postman app	The same collection	Explore, edit, run one request	Import, then ⋯ → Run collection	API on 5080	Not measured (hand run)
Visual Studio	The .NET tests	Debug a test, group by rule	Test → Test Explorer	Nothing	Not measured (hand run)

Runner	Tests	Use when	Command or click path	Needs first	Measured
Harness self-test	221 guard-hook tests	After changing a hook	`node --test ".claude/hooks/__tests__/*.test.mjs"`	Node	100.2 s

Vitest and Playwright (the web app's unit and end-to-end suites) are covered in the UI guide, and are not run in this one.

Check your understanding

Question	Answer
A3. How do I run every test?	`dotnet test --solution Uinsure.Policies.slnx`: four projects, 637 tests.
A4. How can I see each rule has a test?	The `Rule` trait on tests, and `RuleCoverageTests`, which fails on an untested ID.
D9. How do I run one rule's tests?	`-- --filter-trait "Rule=<id>"`, or Group By Traits in Test Explorer.
D10. What do the hooks run?	Pre-commit: privacy scan and build. Pre-push: Release build, full tests, migration script, harness self-test.
U8. How do I try it without code?	The Postman app or CLI collection, or the Scalar page at `/scalar` in Development.

5. Understand it

Four projects, one direction of dependency, rules in the domain.

The domain holds every rule and uses only the .NET base library. Services orchestrate: load, compute today, call the domain, save. Persistence stores. The Api maps HTTP to a service call and back. Architecture tests enforce the arrows.

▼ In depth

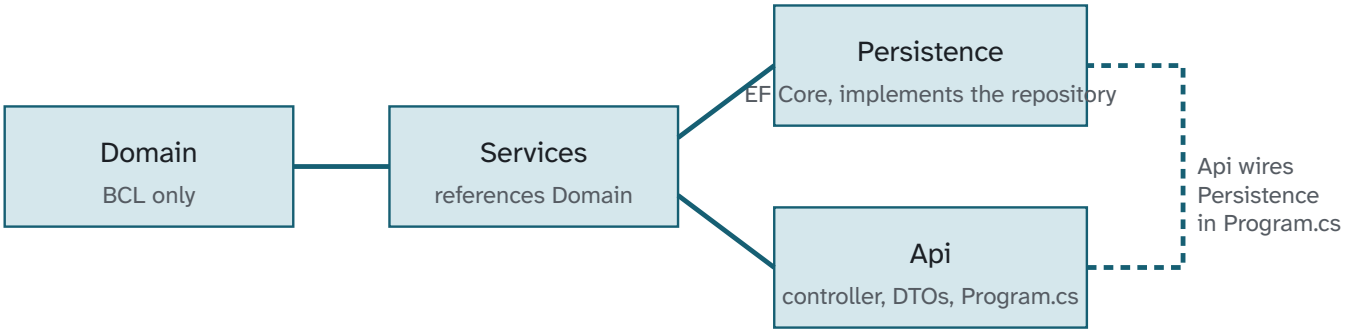


Figure 6. Who may reference whom. An arrow points at what is referenced.

Project	May reference	Must not
Domain	The .NET base library only	Any package, any Uinsure project, `TimeProvider`
Services	Domain	EF Core, ASP.NET Core, Persistence, Api
Persistence	Services, Domain, EF Core	ASP.NET Core, Api
Api	Services, Persistence, Domain	Any test project. Only `Program` touches Persistence

`tests/Uinsure.Policies.Architecture.Tests/LayeringTests.cs:11`

```
    [Fact]
    public void
Domain_types_depend_on_no_Microsoft_namespace_and_not_on_TimeProvider()
    {
        var result = Types.InAssembly(SourceAssemblies.Domain)
            .Should().NotHaveDependencyOnAny("Microsoft", "System.TimeProvider")
            .GetResult();

        Assert.Empty(ArchResults.FailingTypeNamesIn(result,
"Uinsure.Policies.Domain"));
    }
```

A request, from socket to database



Figure 7. The request pipeline, in the order `Program.cs` builds it.

`src/Uinsure.Policies.Api/Program.cs:116`

```
app.UseRateLimiter();
app.UseAuthorization();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
    app.MapScalarApiReference();
}

app.MapControllers();
app.MapHealthChecks("/health/live", new() { Predicate = _ => false
}).DisableRateLimiting();
app.MapHealthChecks("/health/ready", new() { Predicate = check =>
check.Tags.Contains("ready") }).DisableRateLimiting();
```

Before those lines, `Program.cs` applies forwarded headers and the exception handler (src/Uinsure.Policies.Api/Program.cs:83); outside Development it rejects plain HTTP with 400 `/problems/https-required` instead of redirecting. Input is strict: unknown JSON fields and query keys are

400s, and the body is capped at 65,536 bytes. The controller turns a service `Result` into a response, and the service does the orchestration:

src/Uinsure.Policies.Services/PolicyService.cs:12

```
        var today = UkCalendar.Today(clock);

        // SELL-1: a duplicate is a 409 returned alone, before any 422 rule is
evaluated.
        if (await repository.GetByReference(command.Reference, ct) is not null)
        {
            return
Result<PolicyDetails>.Failure(DuplicateReference(command.Reference));
        }
```

One rule, followed: CANCEL-1b

1. **Requirement.** docs/requirements.md:77: "On or after the start date, within the 14-day cooling-off → full refund", interpreted as start + 14 days, inclusive.

2. **Test.** tests/Uinsure.Policies.Api.Tests/CancelCoolingOffTests.cs:22 sells a policy, cancels it on the start date and asserts the refund (extract in section 4). The unit tests sit in `Domain.Tests`.

3. **Code.** src/Uinsure.Policies.Domain/Rules/RefundCalculator.cs:40. One pure method serves both the cancellation and its quote.

   ```
           // CANCEL-1b: cooling-off runs to start + 14 days, inclusive.
           if (cancelOn <= term.Start.AddDays(CoolingOffDays))
           {
               return (RuleIds.Cancel1b, term.Amount);
           }

           // CANCEL-1c: the cancellation day counts as used; divide by the
   term's actual days (365 or 366).
           return (RuleIds.Cancel1c, term.Amount * unusedDays / termDays);
   ```

   You should see: Run it: the trait filter in section 4 ran the 8 tests of this rule, all green.

Time and money

"Today" has one source. Services read the injected `TimeProvider` and convert to Europe/London once; the domain receives a `DateOnly`. An analyser (`BannedSymbols.txt`) turns `DateTime.Now`, `UtcNow` and `Today` into build errors. Money is rounded only in `Money.Round`, and an architecture test allows the analyser suppression in that file alone.

src/Uinsure.Policies.Services/UkCalendar.cs:5

```
    private static readonly TimeZoneInfo London =
TimeZoneInfo.FindSystemTimeZoneById("Europe/London");

    public static DateOnly Today(TimeProvider clock) =>
        DateOnly.FromDateTime(TimeZoneInfo.ConvertTime(clock.GetUtcNow(),
London).DateTime);
```

src/Uinsure.Policies.Domain/Common/Money.cs:5

```
    // CANCEL-6: 2 dp, half away from zero. .NET's default is banker's rounding, so
this is the one place it is chosen.
#pragma warning disable RS0030 // The only permitted call to decimal rounding
(architecture test A11).
    public static decimal Round(decimal amount) => Math.Round(amount, 2,
MidpointRounding.AwayFromZero);
#pragma warning restore RS0030
```

The 29 February case is in the domain too:

src/Uinsure.Policies.Domain/Policies/Term.cs:23

```
    // SELL-3: start + 1 year - 1 day; a 29 Feb start has no anniversary, so it ends
on 28 Feb.
    public static DateOnly EndFor(DateOnly start) =>
        start is { Month: 2, Day: 29 }
            ? new DateOnly(start.Year + 1, 2, 28)
            : start.AddYears(1).AddDays(-1);
```

Persistence, logging and health

The aggregate is loaded whole. `Reference` has a unique index, so even a race on SELL-1 ends in a 409. `Version` is a concurrency token, set anew on every save of a changed policy; two writes on one policy give a 409 `/problems/concurrency`. Money is stored with precision 18,2 and enums as strings. The code never orders or sums money in SQL, because SQLite stores decimals as text.

src/Uinsure.Policies.Persistence/Configurations/PolicyConfiguration.cs:14

```
        // SELL-1: the stored reference is already normalised, so a plain unique
index is case-insensitive in effect.
        builder.Property(p => p.Reference).HasMaxLength(20).IsRequired();
        builder.HasIndex(p => p.Reference).IsUnique();

        builder.Property(p => p.Type).HasConversion<string>().HasMaxLength(20);
        builder.Property(p => p.Version).IsConcurrencyToken();
```

Logs use message templates, with the reference, rule IDs, outcome and trace ID. They never carry names, dates of birth, addresses, postcodes or request bodies. Application Insights is wired in only when

`APPLICATIONINSIGHTS_CONNECTION_STRING` is set. `/health/live` runs no checks; `/health/ready` runs the database check.

Adding a rule or an endpoint

1. Write the row in `docs/requirements.md` with an ID and its interpretation.
2. Write a red test with `[Trait("Rule", "<id>")]`, at HTTP level, then at domain level.
3. Add the domain method and its `RuleIds` entry, then the `ProblemCatalog` entry for its status and title.
4. Add the controller action. Run the architecture tests; they fail if a layer arrow is wrong.

To check the layering part of that, run the architecture tests alone:

```
dotnet test --project tests/Uinsure.Policies.Architecture.Tests
```

The run gave:

```
total: 67
failed: 0
succeeded: 67
duration: 1s 482ms
```

Why, in six decisions

> **Why**
> Terms, not overwritten dates. The alternative was to move the policy's dates on renewal. Renewal happens while the old term still runs, so overwriting would break a cancellation made in its last 30 days (`RENEW-4`).

> **Why**
> One result type and one status mapping. The alternative was exceptions per rule, with status codes chosen in controllers. A `Result` lets the domain report every broken rule at once, and `ProblemCatalog` is the only place a status is chosen.

> **Why**
> Interfaces only at the layer seams. The alternative was an interface per class, a generic repository and a mediator. Three seams (service, repository, reference generator) are what the tests need; more would be speculation.

> **Why**
> No `DateTime.Now`. The alternative was to read the clock where needed. One injected clock makes every date boundary testable, and the build fails on the banned calls.

> **Why**
> SQLite locally. The alternative was SQL Server in a container. SQLite needs nothing installed, so the reviewer runs the API in a minute; the provider switch is a configuration key.

> **Why**
> Rounding in one method. .NET rounds halves to even by default. The brief's pro-rata refunds need half away from zero, so it is chosen once in `Money.Round` (`CANCEL-6`).

Check your understanding

Question	Answer
D1. Who may reference whom?	Domain (base library only) ← Services ← Persistence and Api. Architecture tests enforce it.
D2. What happens to a request?	Pipeline in `Program.cs`, controller, service (load, today, domain, save), repository.
D3. How do I follow a rule?	Requirement row, traited tests, then the domain method: CANCEL-1b is docs/requirements.md:77, tests/Uinsure.Policies.Api.Tests/CancelCoolingOffTests.cs:22, src/Uinsure.Policies.Domain/Rules/RefundCalculator.cs:40.
D4. How is "today" decided?	`UkCalendar.Today(TimeProvider)` in Europe/London; an analyser bans the rest.
D5. How is money rounded?	Only `Money.Round`, 2 dp away from zero.
D7. How do I add a rule?	Requirement row, red test, domain method, catalog entry, controller action.
A10. Which design choices must I defend?	The six "Why" callouts above.

6. Ship it

A Bicep template and an Azure DevOps pipeline, built and checked locally and never deployed.

They show how this API would reach Azure. Nothing here was connected to Azure or Azure DevOps, and `az` was never run. The repository's real gate is its local git hooks. The only thing run for this guide was the Bicep build.

▼ In depth

> **Evidence only**
> No service connection, environment or approval exists. A what-if was not run, because it needs a signed-in subscription.

The Bicep

`infrastructure/main.bicep` works at subscription scope. It creates the resource group `rg-policies-<env>` and calls four modules. Three parameter files give dev, test and prod, in `uksouth`.

Module	Creates
`modules/monitoring.bicep`	Log Analytics workspace, Application Insights
`modules/network.bicep`	Virtual network, two network security groups
`modules/sql.bicep`	Azure SQL server and database, private endpoint, private DNS
`modules/appservice.bicep`	Managed identity, Linux App Service plan, the web app

1. Build the template locally.

   ```
   bicep build infrastructure/main.bicep --outfile main.json
   ```

   You should see: Exit code 0 and no output; the compiled template had 780 lines. The CLI was version 0.48.1.

There are no secrets: the app signs in to Entra-only Azure SQL with a user-assigned managed identity, so no password exists. Key Vault was considered and rejected, because there is nothing to store.

The pipeline

`infrastructure/pipelines/azure-pipelines.yml` triggers on pushes to `main`, with `pr: none`. It runs three groups of work.

Stage	What it does
Build	Build with `-warnaserror`, run the tests, generate the idempotent migration script, publish the app and the infrastructure folder as one artefact
`WhatIf_<env>`	Runs unapproved for dev, test and prod, so the approver can read the what-if
`Deploy_<env>`	Runs the infrastructure deployment, applies the migration script, deploys the zip, smoke-tests `/health/ready`. Prod waits for the approval on the `policies-prod` environment

> **Why**
> Evidence and not a deployment. The alternative was a live deployment. The brief put live deployment out of scope, and an unrun pipeline can claim only what a local check proved: the Bicep builds and the YAML validates against a schema.

Check your understanding

Question	Answer
A8. Is the Azure part real?	Evidence only: Bicep built locally, a pipeline checked against a schema, nothing deployed, `az` never run.
U10. Is it safe to expose?	No authentication exists, and the Bicep sets no WAF or IP restriction. The security scope in `docs/design.md` names the gaps.

7. How it was built

Test first, in a loop, with a recorded trail for every feature.

AI agents in Claude Code built this repository inside a harness, with a human deciding at fixed points. Every rule in the brief got an ID; every feature is a node in a plan; every node leaves a trail of commits, verdicts and findings.

▼ In depth

Idea	What it means here
Outside-in TDD	Red acceptance tests first, then a stub controller, then domain, services, persistence and API turned green layer by layer. Commits carry a `TDD:` trailer (`red` , `stub` , `green` , `refactor`).
The plan DAG	Features are nodes with tasks in `docs/plans/active/build.dag.yaml` . Status changes only by commit.
Six agents	Tester, builder, reviewer, security, investigator and git. The agents that review cannot write, and the ones that write do not judge their own work. The orchestrator runs every gate itself.
The trail	`docs/trace/build/<node>/` : activity, verdicts, findings and a walkthrough.

Counted from `docs/trace/build/` on this branch, the nodes F0a to F6 hold 15 findings (in seven of the nodes), and the reviewer and security verdict files number 51 (F0b 5, F1 4, F1b 4, F2 6, F3 6, F4 6, F5 12, F6 8). A finding sends a node back through the failure route: diagnose, write a new red test, then fix. The third failure stops for a written investigation.

The full story, with each node's walkthrough and every finding, is browsable in Docket, the web app, which the UI guide covers.

Check your understanding

Question	Answer
A6. Was it built test-first? How can I check?	Outside-in TDD. Commits carry `TDD:` trailers, and the trail is under `docs/trace/build/`.

8. Troubleshooting

Symptoms met while building and capturing this guide, and what to do.

Symptom	Cause	Fix
"The filename or extension is too long" in the architecture tests	The clone path is long, and SQLite's native library is copied deep under each test project's `bin`	Clone to a short path such as `C:\src\uinsure-policies`
"address already in use" on `dotnet run`	Visual Studio or another run holds port 5080	Stop the other run
`dotnet run` fails with "Database:Provider must be Sqlite or SqlServer"	It ran with no launch profile, so it was not Development and had no database configuration (met while capturing this guide)	Use `--launch-profile http`
Postman: connection refused	The API is not running, or `baseUrl` is not `http://localhost:5080`	Start the API; select the **local** environment
400 on `GET /policies/{reference}?reference=X`	A query key named like a route parameter is unknown, and unknown keys are refused	Remove the query key
400 `/problems/https-required`	Plain HTTP outside Development	Call the HTTPS address
429 with `Retry-After`	The per-client rate limit	Wait the number of seconds given
422 on a sale that worked yesterday	The start date is now in the past (`SELL-2`)	Send a start date from today to today + 60
409 `/problems/concurrency`	Two writes on one policy at once	Retry the request
The README says the collection has 109 requests; the CLI says 253	109 is the count of test scripts. 253 includes the set-up sales and read-backs	Read the summary table as in section 4
`dotnet ef` warns "tools version '9.0.9' is older than that of the runtime '10.0.13'"	An older global `dotnet-ef`; the repository has no tool manifest	Update the global `dotnet-ef` to the runtime's version. The script is still generated

Symptom	Cause	Fix
A warning in the log about "QuerySplittingBehavior" on the first read	EF Core notes a query that loads several collections. It is a performance note, not a failure	None needed

Check your understanding

Question	Answer
U7. Why did I get a 429 or a 500?	429: too many requests from one address, wait for `Retry-After`. 500: an unexpected server error; quote the `traceId`.